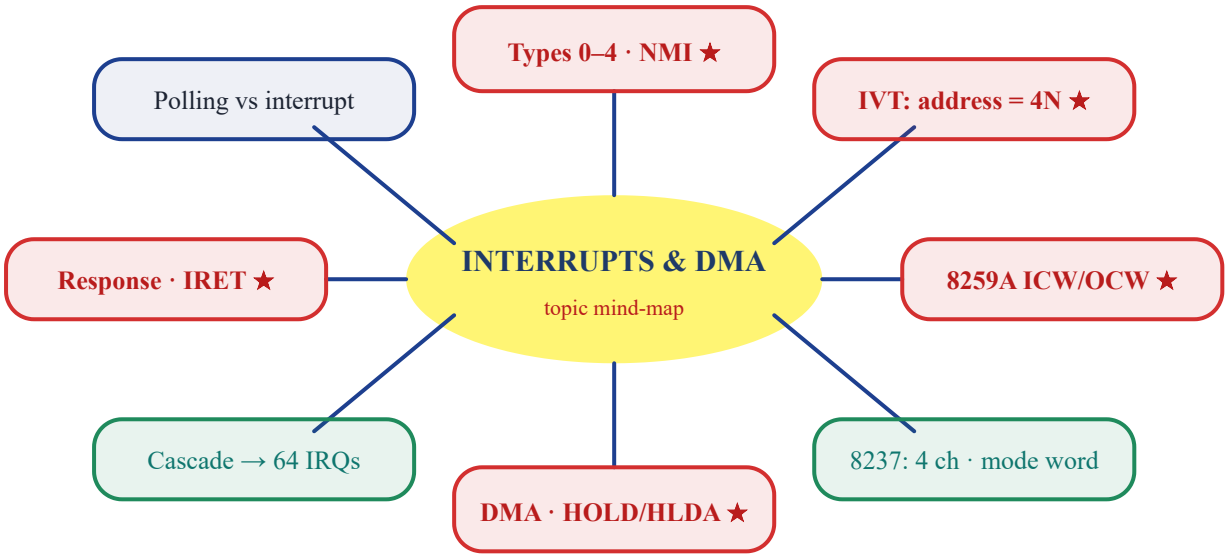
Interrupts, 8259A PIC, DMA & 8237

Types · vector table · 8259A ICW/OCW · DMA · 8237: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = applied / programming

Syllabus coverage

"Types of I/O: … programmed I/O, **interrupt driven I/O** … Interrupts: interrupt driven I/O, **software & hardware interrupts**, **interrupt vectors and vector table**, interrupt processing, **programmable interrupt controller (8259A)**, DMA: **DMA controller (8237)**". The 8086 architecture and flags are in lesson 1; 8255A and I/O decoding in lesson 2; the 8254 timer and 8251 USART in lesson 3.

High-yield list

① vector at 4 × type
② NMI = type 2
③ IVT = 1 KB at 0
④ 2 $\overline{\text{INTA}}$ cycles, IRET
⑤ type = ICW2 + IR
⑥ 1 + 8 PICs → 64
⑦ HOLD → HLDA
⑧ 8237: 4 ch, 64 KB

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

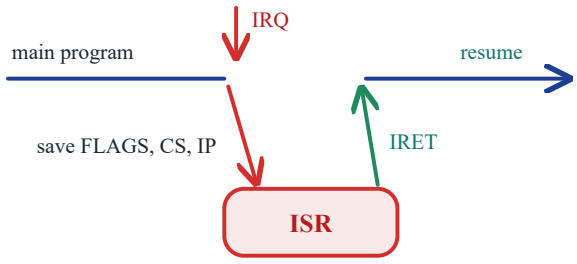
✓ Polling vs interrupts, classes …… p. 2
✓ 8086 types & vector table …… p. 3
✓ Response sequence, IRET, priority … p. 4
✓ 8259A PIC architecture …… p. 5
✓ ICW1–ICW4 …… p. 6
✓ OCWs, modes, cascading …… p. 7
✓ DMA: why, modes, HOLD/HLDA … p. 8
✓ 8237 DMAC …… p. 9
✓ Night-before sheet …… p. 10
✓ Practice MCQs + written …… p. 11

1 Polling vs Interrupt-Driven I/O

> **Interrupt:** A signal (hardware, instruction or CPU-generated) that makes the CPU **suspend the program after the current instruction**, run an **Interrupt Service Routine (ISR)**, then **resume where it stopped**.

Polling (programmed I/O): the CPU keeps reading a status port in a loop until the device is ready → simple, but **wastes CPU time** and a fast event can be missed.

Interrupt-driven I/O: the device **tells** the CPU when it is ready; the CPU does useful work in between. Cost: a little extra hardware and the save/restore overhead (**interrupt latency**).



the ISR is a detour, not a jump

Point	Polling	Interrupt-driven	DMA
Who moves data	CPU	CPU (in the ISR)	**DMAC**, not the CPU
CPU time lost	**highest** (busy-wait)	only during ISR	lowest (bus lent)
Use	simple, slow devices	keyboard, timer, UART	disk, sound, network

> **Memory trick:** Polling = checking your phone every minute · interrupt = it **rings** · DMA = a courier with the store-room key.

2 Classifying Interrupts

Basis	Classes (8086 examples)
Source	**hardware** (NMI, INTR) · **software** (INT n, INT 3, INTO) · **exceptions** (÷ error, single-step)
Masking	**maskable** (INTR: obeys IF, STI/CLI) · **non-maskable** (NMI)
Vector	**vectored**: ISR address fixed (NMI → type 2) · **non-vectored**: source found by polling
Trigger	**edge**-triggered (NMI, rising edge) · **level**-triggered (INTR, held high)

8086 pin	NMI (pin 17)	INTR (pin 18)
Maskable?	**No** (CLI cannot stop it)	Yes: accepted only if **IF = 1**
Trigger	rising edge (high ≥ 2 clocks)	high level until acknowledged
Type number	fixed **type 2** (power fail, parity)	from the 8259A in the 2nd $\overline{\text{INTA}}$

> **Exam trap:** **IF masks only INTR.** NMI, INT n, INT 3, INTO, divide error and single-step all ignore IF. Only INTR produces $\overline{\text{INTA}}$ bus cycles.

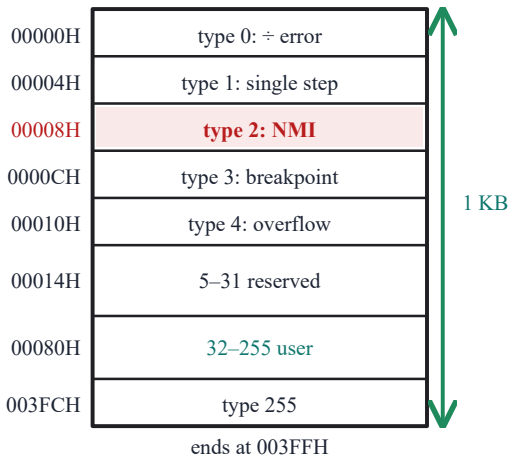
3 The 256 Interrupt Types of the 8086

Type N = 00H–FFH (256): **0–4 dedicated** · 5–31 Intel-reserved · 32–255 user.

Type	Name	Raised when …	Vector at
0	**Divide error**	DIV/IDIV by 0, or the quotient does not fit	00000H
1	**Single step**	TF = 1: after every instruction (debugging)	00004H
2	**NMI**	rising edge on the NMI pin	**00008H**
3	**Breakpoint**	INT 3: the **1-byte** opcode CCH, planted by debuggers	0000CH
4	**Overflow**	INTO executed while OF = 1	00010H

INT n = 2 bytes (CDH, n) · **INT 3** = 1 byte (CCH) · **INTO** = CEH · **IRET** = CFH. No instruction sets TF directly: PUSHF → set bit 8 in the copy → POPF.

4 Interrupt Vector Table (IVT)



IVT at the very bottom of memory

Vector: the 4-byte far address (IP, CS) of an ISR. IVT = 256 × 4 = **1 KB** at **00000H–003FFH**.

Formula: vector address = **4 × N**

IP ← word at 4N (low) · CS ← word at 4N + 2

ISR physical address = CS × 10H + IP



little-endian: offset first, segment second

Worked examples ✍️:

Given	Working	Answer
INT 21H (DOS)	21H × 4 = 84H	**00084H–00087H**
INT 08H (PC timer, IRQ0)	08H × 4 = 20H	**00020H**
vector found at 0002CH	2CH ÷ 4 = 0BH	**type 0BH** (11)
vector found at 000A0H	A0H ÷ 4 = 28H	**type 28H** (40)

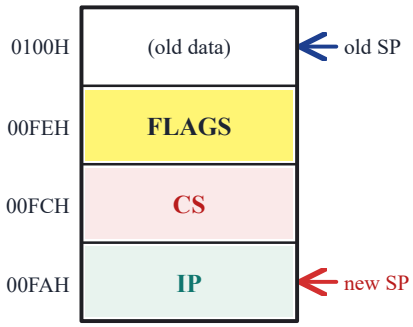
Bytes at 00084H = 56H 34H 00H 12H → IP = 3456H, CS = 1200H → ISR at 12000H + 3456H = **15456H**

Memory trick: × 4 = **shift left 2**: 0010 0001 → 1000 0100 (21H → 84H). "**I** before **C**": IP first, then CS.

Exam trap: The vector of INT 21H is at 84H, **not** at 21H. The type number is an index; the address is 4 times it.

5 What the 8086 Does on an Interrupt

1. Finishes the **current instruction**, then samples NMI and INTR (INTR only if **IF = 1**)
2. **INTR only: two $\overline{\text{INTA}}$ cycles**. 1st: 8259A resolves priority; 2nd: it puts **type N** on D0–D7 (NMI, software: N already known)
3. Pushes **FLAGS** (SP − 2)
4. Clears **IF and TF**: no INTR, no single-step in the ISR
5. Pushes **CS**, then **IP** (SP − 4 more: **6 bytes** in all)
6. Loads **IP from 4N**, **CS from 4N + 2** → ISR runs
7. ISR saves registers, services the device, sends **EOI**, ends with **IRET**
8. **IRET** pops IP, CS, FLAGS (old IF, TF return)



stack after the interrupt is accepted

RET vs IRET

Instruction	Pops	Bytes
RET (near)	IP	2
RETF (far)	IP, CS	4
IRET	IP, CS, **FLAGS**	**6**

```
; ISR skeleton (PC: 8259A at 20H)
ISR:  PUSH AX
      ...              ; service device
      MOV  AL, 20H     ; non-specific EOI
      OUT  20H, AL
      POP  AX
      IRET
```

Memory trick: Push order **"F-C-I"** (Flags, CS, IP), pop order reversed **"I-C-F"**. Interrupt = a far CALL that also saves the flags.

6 Priority, Latency & Nesting

Priority	8086 interrupt
highest	divide error, INT n, INTO
↓	NMI
↓	INTR
lowest	single step

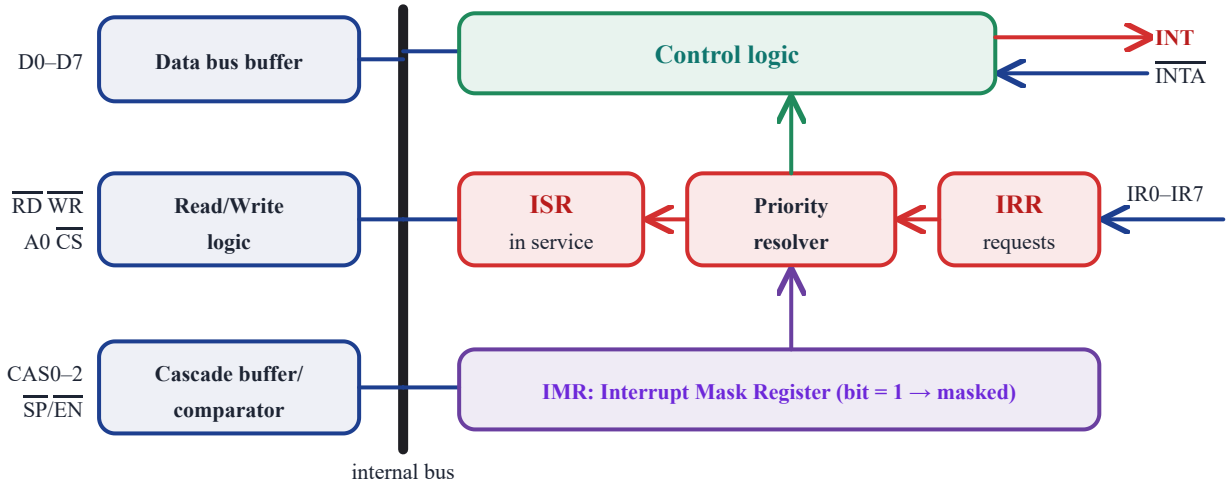
- **Nesting:** IF = 0 inside the ISR; put **STI** in the ISR to let higher-priority INTRs in
- **Latency** = request → first ISR instruction
- After **MOV SS / POP SS** interrupts wait one more instruction
- REP string instructions can be interrupted between iterations

Exam trap: Interrupts are recognised at the end of an **instruction**, but a HOLD (DMA) request is granted at the end of the current **bus cycle**. And RET in place of IRET leaves FLAGS on the stack: the program crashes.

7 8259A Programmable Interrupt Controller

Why a PIC?: The 8086 has **one** maskable input (INTR). The 8259A takes **8 requests (IR0–IR7)**, masks and prioritises them, raises INTR and hands the CPU the winner's **type number**.

- 28-pin DIP, single +5 V, works with 8085 and 8086/88
- **8 levels**; cascade 1 master + 8 slaves → **64 levels**
- Each IR can be **masked** separately (IMR)
- Edge- or level-triggered inputs (LTIM bit)
- Priority modes: fully nested, rotating, special mask, poll
- Programmed with **4 ICWs** (init) and **3 OCWs** (run time)



8259A block diagram: request → mask → priority → in-service → INT

Register	Holds	Set / cleared by
IRR (request)	IR lines now requesting	set by IR input; cleared on 1st $\overline{\text{INTA}}$
ISR (in-service)	levels being serviced	set on 1st $\overline{\text{INTA}}$; cleared by **EOI**/AEOI
IMR (mask)	masked levels (1 = masked)	written by OCW1
Priority resolver	highest unmasked IRR bit	mode set by OCW2

One request, step by step (8086 mode)

1. IR3 goes high → **IRR bit 3** set; if unmasked and higher than anything in service, 8259A raises **INT**
2. CPU (IF = 1) answers with the 1st $\overline{\text{INTA}}$ → ISR bit 3 set, IRR bit 3 cleared
3. 2nd $\overline{\text{INTA}}$ → 8259A puts the type byte on D0–D7: **T7–T3 of ICW2 + 011**
4. ISR bit 3 stays set (blocks equal/lower levels) until the ISR sends **EOI**; in AEOI mode it clears itself at the end of the 2nd $\overline{\text{INTA}}$

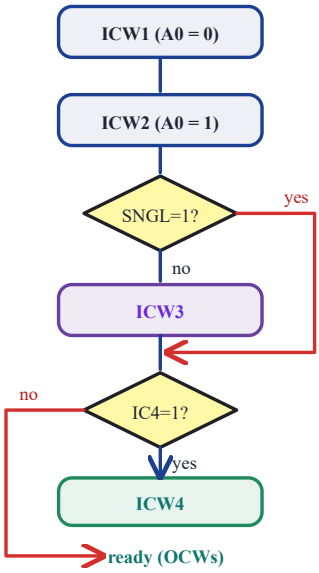
Exam trap: The 8259A sends the **type number** (one byte), not the ISR address. With an 8085 it sends a CALL opcode + 2 address bytes (**3** $\overline{\text{INTA}}$ pulses); with an 8086, **2** pulses.

8 Initialization Command Words (ICW1–ICW4)

One address line, **A0** → two ports (PC: 20H = A0 0, 21H = A0 1). ICWs go **in order**, after reset.

Word	A0	D7	D6	D5	D4	D3	D2	D1	D0
ICW1	0	A7	A6	A5	**1**	LTIM	ADI	SNGL	IC4
ICW2	1	T7	T6	T5	T4	T3	ID2	ID1	ID0
ICW3 master	1	S7	S6	S5	S4	S3	S2	S1	S0
ICW3 slave	1	0	0	0	0	0	ID2	ID1	ID0
ICW4	1	0	0	0	SFNM	BUF	M/$\overline{\text{S}}$	AEOI	µPM

- **ICW1:** D4 = 1 marks it as ICW1 · **LTIM** 1 = level, 0 = edge · **SNGL** 1 = single, 0 = cascade · **IC4** 1 = ICW4 follows (**always 1 for the 8086**) · A7–A5, ADI: 8085 only
- **ICW2:** upper 5 bits of the type; the 8259A fills the lower 3 bits with the IR number → **type = ICW2 + IR no.** (ICW2 is a multiple of 8)
- **ICW3** (only if SNGL = 0): master: 1 in each bit with a slave · slave: its ID = the master IR it is wired to
- **ICW4: µPM** 1 = 8086/88 · **AEOI** 1 = auto EOI · **BUF** buffered mode, then **M/$\overline{\text{S}}$** 1 = master · **SFNM** special fully nested (master in cascade)



Worked words ✍️:

Requirement	Binary	Hex
ICW1: edge, single, ICW4 needed	0001 0011	**13H**
ICW1: edge, cascade, ICW4 needed	0001 0001	**11H**
ICW2: IR0 → type 08H (IR3 → 0BH)	0000 1000	**08H**
ICW4: 8086, normal EOI, non-buffered	0000 0001	**01H**
ICW4: 8086, auto EOI	0000 0011	**03H**
ICW4: 8086, buffered master	0000 1101	**0DH**

```
; single 8259A, ports 20H/21H
MOV AL,13H  ; OUT 20H,AL  ; ICW1
MOV AL,08H  ; OUT 21H,AL  ; ICW2
MOV AL,01H  ; OUT 21H,AL  ; ICW4
MOV AL,0FCH ; OUT 21H,AL  ; OCW1
; IR0, IR1 enabled
```

Memory trick: Order "**1-2-(3)-(4)**": ICW3 only if **cascaded**, ICW4 only if **IC4 = 1** (always, for an 8086).

Exam trap: On an 8086 (16-bit bus) the 8259A sits on D0–D7 and its A0 is wired to CPU **A1**, so its ports are e.g. 80H and **82H**, not 80H/81H.

9 Operation Command Words (OCW1–OCW3)

Word	A0	D7	D6	D5	D4	D3	D2	D1	D0
OCW1 (= IMR)	1	M7	M6	M5	M4	M3	M2	M1	M0
OCW2	0	R	SL	EOI	**0**	**0**	L2	L1	L0
OCW3	0	0	ESMM	SMM	**0**	**1**	P	RR	RIS

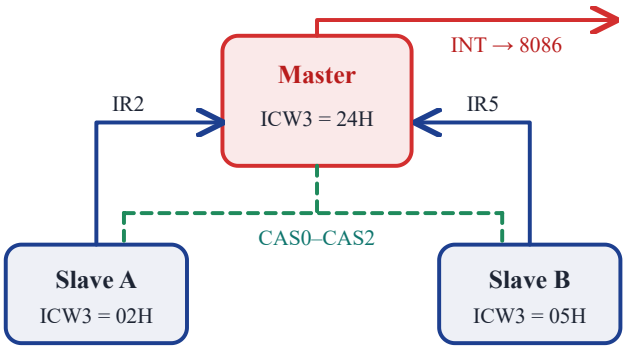
How the chip tells words apart: A0 = 0 & D4 = 1 → ICW1 · A0 = 0, D4 D3 = 00 → OCW2 · A0 = 0, D4 D3 = 01 → OCW3 · A0 = 1 → ICW2/3/4 during init, otherwise **OCW1**.

OCW2 command (R SL EOI)	Word
001 non-specific EOI	**20H**
011 specific EOI (IR3)	**63H**
101 rotate on non-spec. EOI	**A0H**
100 rotate in AEOI (set)	**80H**
110 set priority (IR5 lowest)	**C5H**

OCW3 / OCW1 use	Word
read IRR on next $\overline{RD}$	**0AH**
read ISR on next $\overline{RD}$	**0BH**
poll command	**0CH**
set / reset special mask	**68H / 48H**
OCW1: enable IR0, IR1 only	**FCH**

Priority mode	Rule
Fully nested (default)	IR0 highest … IR7 lowest; ISR blocks equal/lower
Automatic rotation	level just served → **lowest** (IR4 served → IR5 top)
Specific rotation	software sets the lowest (C5H → IR6 highest)
Special mask	lower levels allowed while an ISR runs
Poll	no INT; poll byte D7 = request, D2–D0 = level

10 Cascading: up to 64 Levels



- Slave INT → a master IR input; master **CAS0–2** outputs the slave ID; slaves read it as inputs
- Hardware sets master/slave by **$\overline{SP}$/$\overline{EN}$** (1 = master) or by ICW4 in buffered mode
- On the 2nd $\overline{INTA}$ the **selected slave** puts the type on the bus
- Levels with s slaves = **8 + 7s**: 1 slave → 15 (PC/AT), 8 slaves → **64**
- ISR on a slave level sends **two EOIs**: slave, then master

> **Worked example ✍️:** Master + slaves on IR2 and IR5 (edge, 8086): master ICW1 = **11H** (cascade), ICW2 = 08H, ICW3 = 0010 0100 = **24H**, ICW4 = 01H · slave A: 11H, 70H, ICW3 = **02H**, 01H · slave B: ICW3 = **05H**. Levels = 8 + 7 × 2 = **22**.

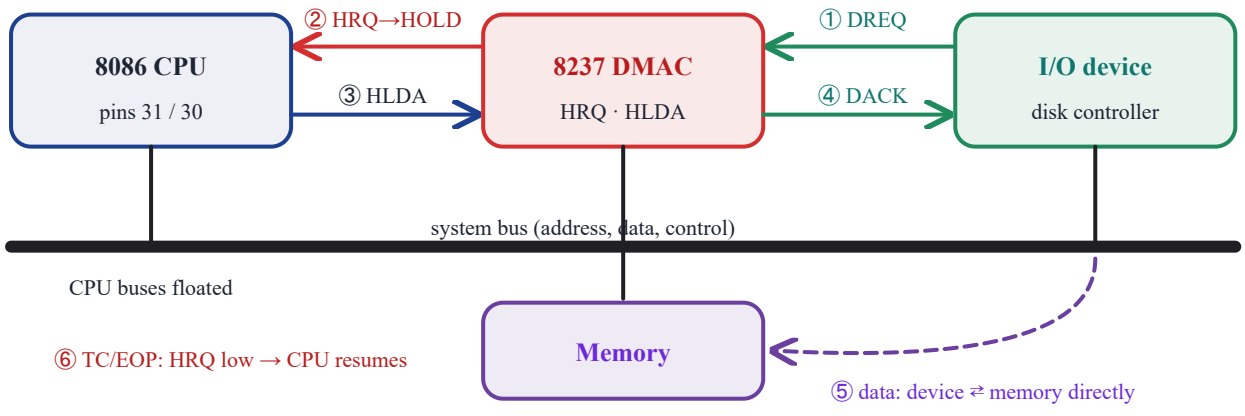
11 Direct Memory Access (DMA)

> **Definition:** DMA moves a block between an I/O device and memory (or memory to memory) **without the CPU running an instruction per byte**: a **DMA controller (DMAC)** borrows the bus, drives addresses and strobes itself, then returns the bus.

Why? Polling/interrupt I/O moves every byte **through the CPU** (IN, MOV, INC, LOOP ≈ tens of clocks). A DMAC needs **one bus cycle per byte** (4 clocks on the 8237).

Cost: an extra chip, and the CPU is **held off the bus** during transfers (it may still run from its queue).

The HOLD / HLDA handshake (8086 minimum mode)



DREQ → HRQ/HOLD → HLDA → DACK → transfer → release

1. Device asks the DMAC with **DREQ**; DMAC raises **HRQ**, wired to the 8086 **HOLD** pin
2. 8086 finishes the current **bus cycle**, floats its address/data/control lines and answers **HLDA**
3. DMAC sends **DACK**, drives the address and $\overline{\text{MEMR}}$ + $\overline{\text{IOW}}$ (or $\overline{\text{IOR}}$ + $\overline{\text{MEMW}}$)
4. At terminal count (or $\overline{\text{EOP}}$) HRQ drops → the 8086 drops HLDA and takes the bus back

DMA mode	How the bus is used	Effect
Burst (block)	whole block in one bus grant	**CPU off the bus**; fastest
Cycle stealing (single)	one byte per grant, bus returned	CPU slowed slightly
Transparent (hidden)	only cycles the CPU leaves idle	no slowdown; slowest

Memory trick: **Burst** = take the road for the whole convoy · **stealing** = slip in one car at a time · **transparent** = drive only when the road is empty.

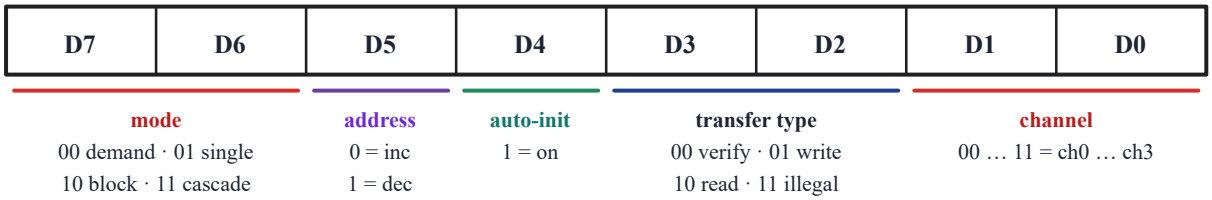
Exam trap: HOLD is honoured even when IF = 0 (it is not an interrupt). In **maximum mode** HOLD/HLDA are replaced by $\overline{\textbf{RQ}}$/**GT0** (higher priority) and $\overline{\textbf{RQ}}$/**GT1**.

12 8237 Programmable DMA Controller

- 40 pins, **4 channels** (DREQ/DACK 0–3)
- 16-bit address + count → **64 KB per block**
- 8237A-5: 5 MHz, up to **1.6 MB/s**
- Fixed (ch0 highest) or rotating priority
- **A8–A15 sent on DB0–DB7**, latched by **ADSTB**
- **AEN** cuts off the CPU address latches
- $\overline{\text{EOP}}$: out at TC, or in to stop a transfer
- PC/AT: 2 cascaded → **7** usable channels

Register	Bits × no.	Role
Base / current address	16 × 4 each	start / next address
Base / current count	16 × 4 each	load **N − 1**; TC at 0000H → FFFFH
Command	8 × 1	enable, mem-to-mem, priority, timing
Mode	6 × 4	mode, type, inc/dec, auto-init
Mask · request · status · temp	4 · 4 · 8 · 8	disable ch · soft DREQ · TC flags · mem-to-mem

Mode register (one per channel)



Write = I/O → **memory** ($\overline{\text{IOR}}$ + $\overline{\text{MEMW}}$) · **Read** = **memory** → I/O ($\overline{\text{MEMR}}$ + $\overline{\text{IOW}}$) · **Verify** = no strobes. Named from the **memory's side**.

Worked mode words ✍️:

Requirement	D7…D0	Hex
ch2, single, inc, no auto-init, **write** (disk → memory)	01 0 0 01 10	**46H**
ch2, single, inc, no auto-init, **read** (memory → disk)	01 0 0 10 10	**4AH**
ch1, block, inc, auto-init, read	10 0 1 10 01	**99H**
ch3, cascade (second 8237 hangs here)	11 0 0 00 11	**C3H**

1 KB on ch1: count = 1024 − 1 = **03FFH**; clear the byte flip-flop, write address and count (low, high), mode 45H, unmask ch1.

Memory trick: Mode word = "**M-A-A-T-C**": Mode (2), Address dir (1), Auto-init (1), Type (2), Channel (2).

Exam trap: 16-bit address → a block cannot cross a **64 KB page**; the PC adds a **page register** for A16–A19. Mem-to-mem uses **ch0 → ch1** only.

Night-Before-the-Exam Sheet

Fact sheet:

256 types · IVT 00000H–003FFH = 1 KB

Vector at 4N: IP at 4N, CS at 4N + 2

0 ÷ error · 1 step · 2 NMI · 3 break · 4 INTO

0–4 dedicated · 5–31 reserved · 32–255 user

Push FLAGS · clear IF, TF · push CS, IP

IRET pops IP, CS, FLAGS

Priority: ÷/INT n/INTO > NMI > INTR > step

INTR: IF = 1 needed, 2 $\overline{\text{INTA}}$ cycles

8259A: 8 IRs, 28 pins, 4 ICW + 3 OCW

Type = ICW2 (T7–T3) + IR number

ICW1 13H · ICW4 01H · OCW2 EOI 20H

IRR → priority → ISR; IMR = OCW1

Levels = 8 + 7s → 64 with 8 slaves

DMA: DREQ → HOLD → HLDA → DACK

Burst · cycle stealing · transparent

8237: 4 ch · 64 KB · count = N − 1

Top 12 one-liners for MCQs

1. NMI = type 2, vector at 00008H
2. INT 3 is the only 1-byte INT (CCH)
3. CLI/STI affect INTR only
4. TF = 1 → type 1 after each instruction
5. INT 21H vector at 00084H
6. 8259A gives the type, not the address
7. ICW3 only when cascaded
8. Default mode: fully nested, IR0 highest
9. AEOI: ISR bit cleared by the 2nd $\overline{\text{INTA}}$
10. HOLD granted after the current bus cycle
11. Write transfer = I/O → memory
12. Max-mode DMA pins: $\overline{\text{RQ}}$/$\overline{\text{GT0}}$, $\overline{\text{RQ}}$/$\overline{\text{GT1}}$

Top traps: type number ≠ vector address · IF does not mask NMI or software INTs · IRET ≠ RET · 8086 needs IC4 = 1 and µPM = 1 · a slave-level ISR needs two EOIs · 8237 count register = bytes − 1 · read/write transfers are named for memory · a DMA block cannot cross a 64 KB page

Compare: 8085: 5 hardware interrupts: **TRAP** (non-maskable, 0024H) > RST 7.5 (003CH) > RST 6.5 (0034H) > RST 5.5 (002CH) > INTR (non-vectored). Vector = RST number × 8.

All tricks together: phone check / ring / courier · × 4 = shift left 2 · "I before C" in the IVT · push F-C-I, pop I-C-F · ICW order 1-2-(3)-(4) · burst / steal / transparent = convoy / one car / empty road · mode word M-A-A-T-C



from an IR pin to the IVT in three moves

✓ Practice MCQs (exam pattern, 10 marks)

1. The vector of INT 13H begins at:
(a) 00013H (b) 0004CH
(c) 00026H (d) 00130H

2. The 8086 NMI has type number:
(a) 0 (b) 1
(c) 3 (d) 2

3. Size of the 8086 interrupt vector table:
(a) 1 KB (b) 256 bytes
(c) 64 KB (d) 4 KB

4. An ISR must end with:
(a) RET (b) IRET
(c) RETF (d) HLT

5. How many $\overline{\text{INTA}}$ cycles does the 8086 run for an INTR?
(a) 1 (b) 3
(c) 2 (d) 0

6. Levels with 1 master + 8 slave 8259As:
(a) 64 (b) 72
(c) 56 (d) 16

7. The OCW2 word for a non-specific EOI is:
(a) 60H (b) A0H
(c) 0BH (d) 20H

8. A DMAC requests the bus (min mode) on:
(a) INTR (b) HLDA
(c) HOLD (d) READY

9. 8237 mode: ch2, single, inc, write:
(a) 46H (b) 4AH
(c) 42H (d) 56H

10. DMA mode using only idle bus cycles:
(a) burst (b) cycle stealing
(c) block (d) transparent

Answer key → 1-b · 2-d · 3-a · 4-b · 5-c · 6-a · 7-d · 8-c · 9-a · 10-d

Written question: A single 8259A at ports 20H/21H serves an 8086 system. Initialise it for edge-triggered inputs, IR0 = type 40H and normal EOI, then enable only IR0–IR3. Give each control word and the code, and find the IVT address used by IR3. 8 marks

1. ICW1 = 0001 0011 = **13H** (edge, single, ICW4 needed) → port 20H 2
2. ICW2 = **40H** → port 21H (IR0–IR7 = types 40H–47H); no ICW3, since single 1
3. ICW4 = 0000 0001 = **01H** (8086 mode, normal EOI, non-buffered) → port 21H 1

4. OCW1 = 1111 0000 = **F0H** → 21H (IR4–7 masked); each: MOV AL,w / OUT p,AL 2
5. IR3 → type 43H → 43H × 4 = **0010CH–0010FH** (IP at 0010CH, CS at 0010EH); its ISR ends with MOV AL,20H / OUT 20H,AL / IRET 2

Before the exam ✓

- ✓ Types 0–4 and their vectors
- ✓ Type ↔ IVT address, both ways
- ✓ INTR response sequence in order
- ✓ ICW1/2/4, OCW1/2 in binary
- ✓ HOLD/HLDA and the 8237 mode word

Also practise (written):

① ICWs: slaves on IR2, IR6
② 8259A block diagram
③ Polling vs interrupt vs DMA
④ 8237 for a 4 KB disk read